Lab 3: Parallel Image Processing with OpenMP
Course: Parallel Computing
Time Limit: 120 Minutes
Goal: Master the **Map** and **Stencil** parallel patterns by implementing real-world image filters.

Part 1: Conceptual Background (Read This First)

Before writing code, you must understand how images exist in computer memory and why parallelizing them requires specific strategies.

1. The Challenge: 2D Images in 1D Memory

When you view an image, you see a 2D grid of pixels (Rows x Columns). However, computer memory is linear (1D). To store a 2D image, we place the rows one after another in a long line. This is called **Row-Major Order**.

Additionally, a standard colour pixel is not a single number; it is a combination of three "channels": **Red, Green, and Blue (RGB)**. These are often interleaved.

- **Logical View:** Pixel(0,0) is a single dot.
- **Memory View:** [R, G, B] are three consecutive bytes.

The Indexing Formula:

To find the memory address of the pixel at (x, y), we use this formula:

index = (y x Width + x) x 3

- **y x Width**: Skips y full rows of data.
- **+ x**: Moves x pixels into the current row.
- **x 3**: Accounts for the 3 bytes (R, G, B) per pixel.

2. The "Map" Pattern (Embarrassingly Parallel)

The simplest parallel operation is the **Map Pattern**. This happens when a function is applied to every element of a collection **independently**.

- **Example:** Inverting colours. To calculate the new colour of Pixel A, you only need to look at Pixel A.
- **Parallelism:** Thread 1 can process the top half, and Thread 2 can process the bottom half. They never need to talk to each other. This is safe and easy.

3. The "Stencil" Pattern (Neighbourhood Dependencies)

Many advanced algorithms (Blur, Edge Detection, Sharpen) use a **Stencil**. This means the value of a pixel depends on its **neighbours**.

- **Example:** A "Box Blur" sets a pixel's value to the average of the 8 pixels surrounding it.

- **The Danger (Race Conditions):** If you update the image "in-place" (writing the new blurred value directly over the old value), the next thread will read that *new* blurred value instead of the *original* value when it calculates the neighbour. This corrupts the data.
- **The Solution (Double Buffering):** We use two arrays:
    1. **Source:** Read-Only (Old values).
    2. **Destination:** Write-Only (New values).

Part 2: Lab Setup

Since we are on lab computers, we cannot install image viewers in VS Code. We will use a custom C program that generates its own test images (with noise) and saves them as **Standard Bitmaps (.BMP)**, which open on any computer natively.

Step 1: Create a file named lab3_image.c.

Step 2: Copy the following code exactly.

(Note: I have pre-written the writeBMP function for you because the binary headers are complex. Your job is to focus on the processing logic in main.)

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

#define WIDTH 1920
#define HEIGHT 1080
#define CHANNELS 3 // Red, Green, Blue

typedef struct {
    int width;
    int height;
    unsigned char *data; // The 1D array of pixel data
} Image;

// Allocates memory for a new image
Image* create_image(int w, int h) {
    Image *img = (Image*)malloc(sizeof(Image));
    img->width = w;
```

```
    img->height = h;
    img->data = (unsigned char*)malloc(w * h * CHANNELS);
    return img;
}

// Generates a smooth gradient with random "salt and pepper" noise.
// We add noise so we can visually see the Blur filter working later.
void generate_noisy_image(Image *img) {
    for (int y = 0; y < img->height; y++) {
        for (int x = 0; x < img->width; x++) {
            int index = (y * img->width + x) * CHANNELS;

            // Background Gradient
            img->data[index] = (unsigned char)(x % 255);      // Red
            img->data[index+1] = (unsigned char)(y % 255);    // Green
            img->data[index+2] = 128;                         // Blue

            // Add Noise (Random white specks)
            if ((rand() % 100) < 5) { // 5% chance
                img->data[index] = 255;
                img->data[index+1] = 255;
                img->data[index+2] = 255;
            }
        }
    }
}

// Save as BMP (Windows Bitmap) so you can view it without installing software.
void writeBMP(const char *filename, Image *img) {
    FILE *f = fopen(filename, "wb");
    if (!f) { perror("File open failed"); return; }
```

```
    int w = img->width, h = img->height;

    // Standard BMP Header
    unsigned char fileHeader[14] = { 'B','M', 0,0,0,0, 0,0, 0,0, 54,0,0,0 };
    unsigned char infoHeader[40] = { 40,0,0,0, 0,0,0,0, 0,0,0,0, 1,0, 24,0 };

    int padSize  = (4 - (w * 3) % 4) % 4;
    int size = 54 + (w * 3 + padSize) * h;

    fileHeader[2] = (unsigned char)(size); fileHeader[3] = (unsigned char)(size>>8);
    fileHeader[4] = (unsigned char)(size>>16); fileHeader[5] = (unsigned char)(size>>24);
    infoHeader[4] = (unsigned char)(w); infoHeader[5] = (unsigned char)(w>>8);
    infoHeader[6] = (unsigned char)(w>>16); infoHeader[7] = (unsigned char)(w>>24);
    infoHeader[8] = (unsigned char)(h); infoHeader[9] = (unsigned char)(h>>8);
    infoHeader[10] = (unsigned char)(h>>16); infoHeader[11] = (unsigned char)(h>>24);

    fwrite(fileHeader, 1, 14, f);
    fwrite(infoHeader, 1, 40, f);

    unsigned char padding[3] = {0,0,0};
    for (int y = h - 1; y >= 0; y--) {
        for (int x = 0; x < w; x++) {
            int i = (y * w + x) * 3;
            // BMP expects BGR format (Blue, Green, Red)
            fputc(img->data[i+2], f); fputc(img->data[i+1], f); fputc(img->data[i], f);
        }
        fwrite(padding, 1, padSize, f);
    }
    fclose(f);
    printf("Saved %s\n", filename);
}
```

```
int main() {
    Image *source = create_image(WIDTH, HEIGHT);
    generate_noisy_image(source);
    writeBMP("original.bmp", source);

    // TODO: Call your filters here...

    free(source->data);
    free(source);
    return 0;
}
```

Step 3: Compile and run.

```
gcc -fopenmp lab3_image.c -o lab3
./lab3
```

Action: Open original.bmp (Double-click). You should see a colorful gradient covered in white static/noise.

Part 3: The Map Pattern - "Invert"

Theory:

Inverting a color creates its "negative." In the 8-bit color space (0–255), the math is:

NewValue = 255 - OldValue

Why it's Easy:

Because calculating pixel i does not require checking pixel i-1 or i+1, we can treat the entire image as one massive 1D array of bytes. We don't even need to know x or y.

Task:

Implement the invert_parallel function.

Update Main:

```
    invert_parallel(source);
    writeBMP("inverted.bmp", source);
```

Verify: Open inverted.bmp. The colors should be flipped, and the white noise dots should turn black.

Part 4: The Stencil Pattern - "Box Blur"

Theory:

A Box Blur smooths an image by taking the average of a 3 x 3 grid around every pixel.

$$Pixel_{new} = \frac{\sum neighbours}{9}$$

The Constraints:

1. **Boundaries:** We cannot blur the pixels on the very edge (row 0 or column 0) because they don't have neighbors on all sides. Our loop must start at 1 and end at Height-1.
2. **Dependencies:** We must use **Double Buffering** (Read from Source, Write to Dest) to avoid race conditions.

Task:

Implement the blur_parallel function.

Heavy Load Test (Iterative Stencil):

A single blur is too fast to measure accurately. We will run it **20 times** to simulate a heavy workload (and make the visual smoothing obvious).

Update Main:

```
// 1. Reset source to the noisy original
generate_noisy_image(source);

// 2. Create a second buffer for Double Buffering
Image *buffer = create_image(WIDTH, HEIGHT);

// 3. Run the "Ping-Pong" loop
// We swap Source and Buffer every iteration so the output of one becomes input of next
double start = omp_get_wtime();
for (int i = 0; i < 20; i++) {
    if (i % 2 == 0) {
        blur_parallel(source, buffer); // Read Source -> Write Buffer
    } else {
        blur_parallel(buffer, source); // Read Buffer -> Write Source
    }
}
```

```
    double end = omp_get_wtime();
    printf("Heavy Blur Time: %f seconds\n", end - start);

    // 4. Save result (It is in 'source' because loop ended on odd iteration 19 -> back to source)
    writeBMP("blurred_heavy.bmp", source);

    free(buffer->data); free(buffer);
```

Verify: Open blurred_heavy.bmp. The white noise specks should be "smudged" out and barely visible.

Part 5: Performance Experiments

Task:

Run your Heavy Blur application with different numbers of threads and record the time.

Part 6: Lab Report (Submit to D2L)

Submit your code (lab3_image.c) and a short text file that includes the **Speedup Analysis**.